

Base de Datos No SQL

SEMANA 15

Heider Sanchez
hsanchez@utec.edu.pe

4.



Base de Datos NoSQL

BD de Columna Ancha



BD de Columna Ancha

Características:

- Inspirado en el modelo **BigTable de Google**.
- Datos organizados en columnas en lugar de filas tradicionales.
 - Esquema flexible, permite que determinadas columnas solo se apliquen a determinados registros.
 - Las filas se identifican por una clave primaria, que puede ser simple o compuesta.
- Escalamiento horizontal y procesamiento distribuido para Big Data.
- Alta disponibilidad: Datos replicados automáticamente entre nodos.

• Table with single-row partitions

Diagram illustrating a table with single-row partitions. The table is organized by columns (performer, born, country, died, founded, style, type) and rows (John Lennon, Paul McCartney, The Beatles). The partition key is the performer. The columns are labeled as columns, and the rows are labeled as rows. The cells are labeled as cells.

performer	born	country	died	founded	style	type
John Lennon	1940	England	1980		Rock	artist
Paul McCartney	1942	England			Rock	artist
The Beatles		England		1957	Rock	band

• Column family view

Diagram illustrating the column family view. The table is organized by rows (John Lennon, Paul McCartney, The Beatles) and columns (born, country, died, founded, style, type). The rows are labeled as rows, and the columns are labeled as columns. The cells are labeled as cells.

	born	country	died	founded	style	type
John Lennon	1940	England	1980		Rock	artist
Paul McCartney	1942	England			Rock	artist
The Beatles		England		1957	Rock	band

BD de Columna Ancha

Agregar Registro con Nueva Columna

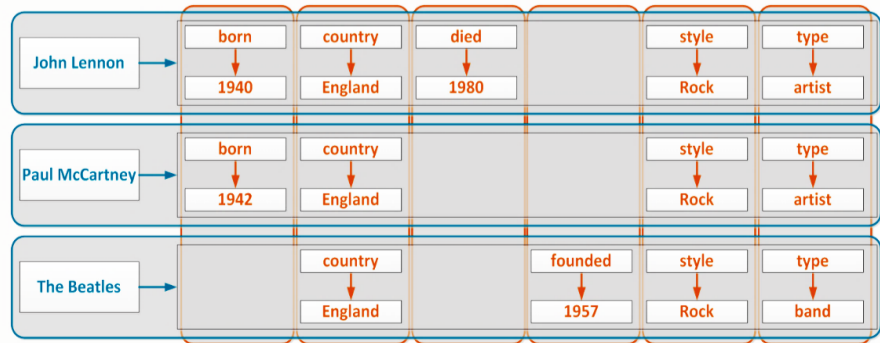
- En una BD Relacional agregar una nueva columna implica completar con valores nulos las demás filas.
- En una BD de Columna Ancha, se agrega elementos a la nueva columna sin afectar los datos existentes.

Table with single-row partitions

Diagram illustrating a table with single-row partitions. The table has columns: performer, born, country, died, founded, style, and type. The rows represent partitions: John Lennon, Paul McCartney, and The Beatles. The 'founded' column is highlighted in red, indicating it is the new column being added. Arrows point from the labels 'columns', 'partition key', 'partitions', 'rows', and 'cells' to their respective parts in the diagram.

performer	born	country	died	founded	style	type
John Lennon	1940	England	1980		Rock	artist
Paul McCartney	1942	England			Rock	artist
The Beatles		England		1957	Rock	band

Column family view



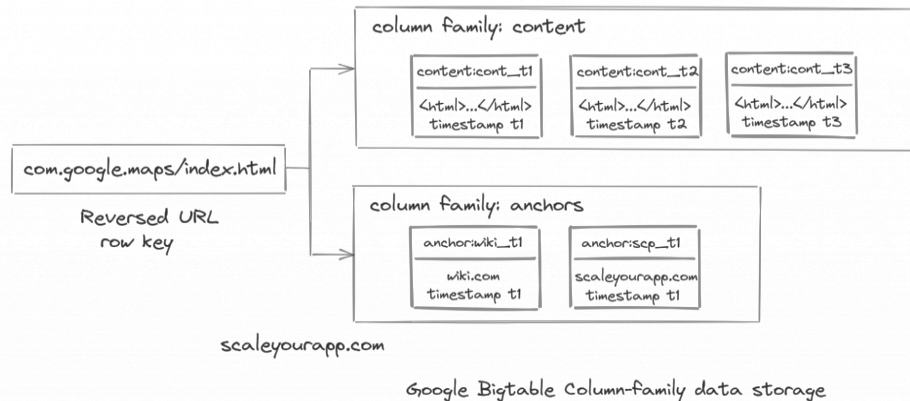
BD de Columna Ancha

¿Por qué se llama BD de Columna Ancha?

- *Cada fila puede contener un numero muy grande de columnas.*

¿Qué es una Familia de Columnas (Column Family)?

- Es una agrupación de columnas que comparten una misma clave de fila.
- En GibTable y Hbase, Column Family es un grupo lógico explicito de columnas dentro de la tabla.
- En Cassandra, Column Family es equivalente a una tabla.



Keys	Column families		
a	colA:value1	colFoo:a value	fram:zilk
b	colA:value1	colB:a value	♟: chesspiece
bb	colA:value1	colB: colFoo:a value	🎵: 🎶
c	colA:Ⓜ	colBaz:anything	colFoo:a value

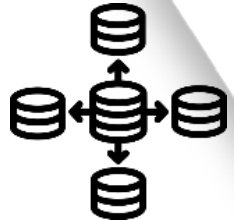
BD de Columna Ancha: Productos

Base de Datos	Año de Creación	Empresa que la Soporta	Características Clave	Casos de Uso
Google Bigtable	2005	Google	Análisis en tiempo real, almacenamiento masivo, IoT.	Análisis predictivo, personalización, servicios IoT.
Apache Cassandra	2008	Apache Software Foundation	Escalabilidad horizontal, alta disponibilidad, Big Data.	Monitoreo en tiempo real, IoT, aplicaciones financieras.
HBase	2008	Apache Software Foundation	Integración con Hadoop, consistencia fuerte, Big Data.	Almacenamiento para análisis de datos, log de eventos.
ScyllaDB	2015	ScyllaDB Inc.	Compatibilidad con Cassandra, baja latencia, alto rendimiento.	Aplicaciones de baja latencia, sistemas de recomendación.

BD de Columna Ancha

Casos de Uso:

- **Análisis en tiempo real:** Procesamiento rápido de grandes volúmenes de datos.
- **Aplicaciones IoT:** Gestión de datos de sensores y dispositivos conectados.
- **Gestión de contenido:** Almacenamiento de datos no estructurados como catálogos.
- **Sistemas de mensajería:** Soporte para chats y redes sociales.
- **Big Data distribuido:** Consultas paralelas para análisis de datos.



BD de Columna Ancha: Apache Cassandra

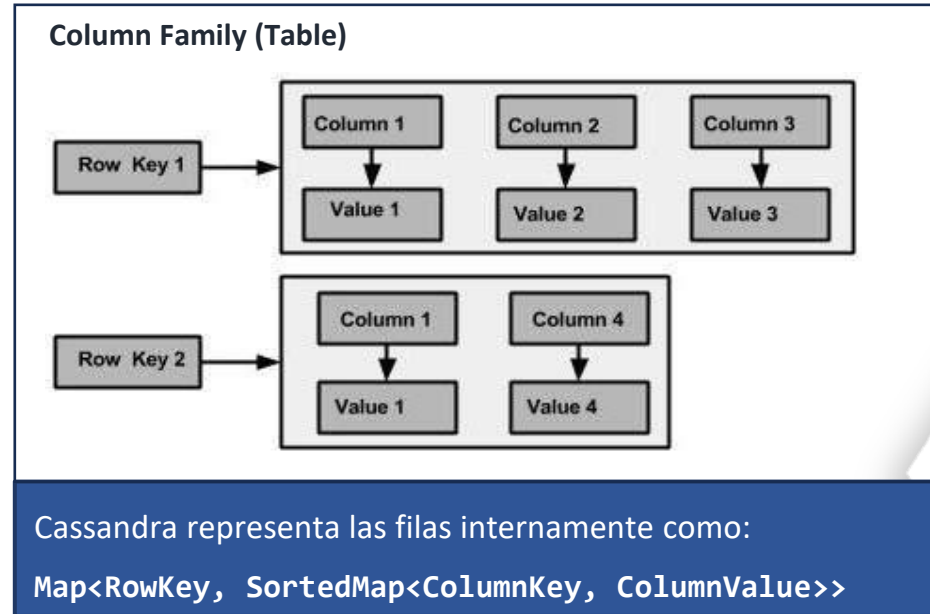


- Inicialmente desarrollado por Facebook
- En Cassandra, los datos se organizan en tablas dentro de keyspaces.
- Utiliza CQL como lenguaje de consulta para interactuar con los Keyspaces.
- **Características clave:**
 - **Alta disponibilidad:** Diseñada para garantizar que los datos estén accesibles incluso si fallan algunos nodos (replicación).
 - **Escalabilidad horizontal:** Puede añadir nodos al clúster fácilmente para manejar más datos y tráfico (utiliza P2P).

BD de Columna Ancha: Apache Cassandra

MODELO DE DATOS

- **Columna:** Un par nombre/valor.
- **Fila:** Un contenedor de columnas identificado por una clave primaria.
- **Tabla:** Un contenedor de filas que están organizadas por particiones.
- **Partición:** Una colección de filas relacionadas que se almacenan juntas en los mismos nodos.



BD de Columna Ancha: Apache Cassandra

MODELO DE DATOS

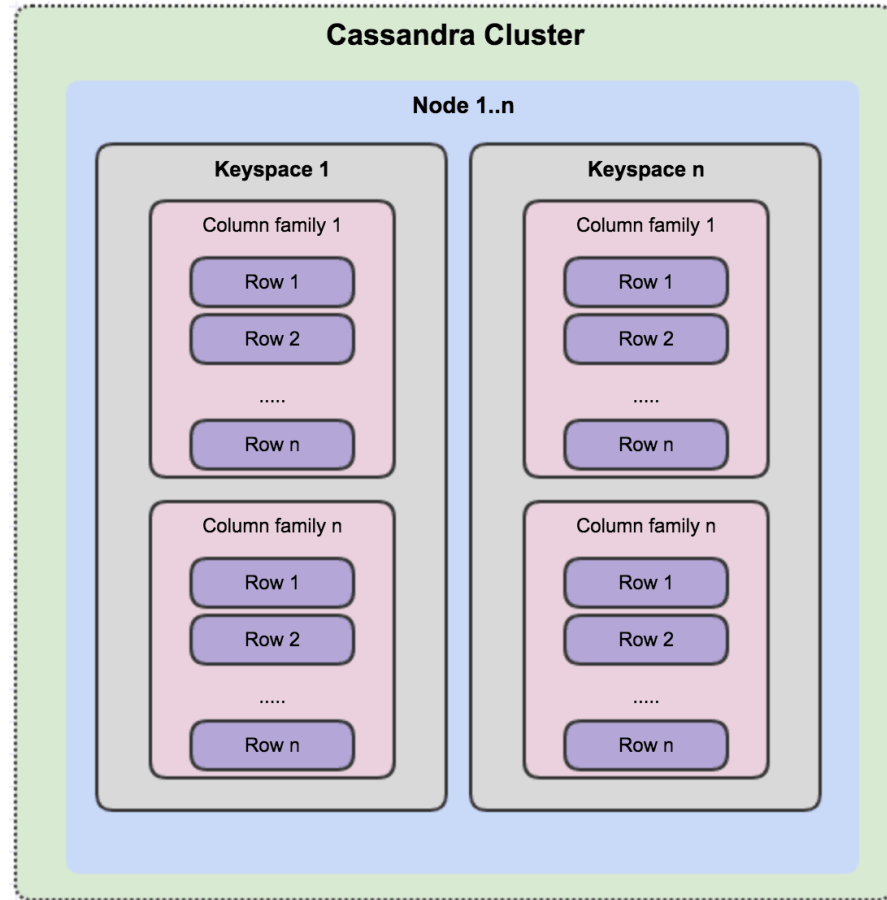
Keyspace

Es un contenedor de tablas
(Column Family)

Cluster

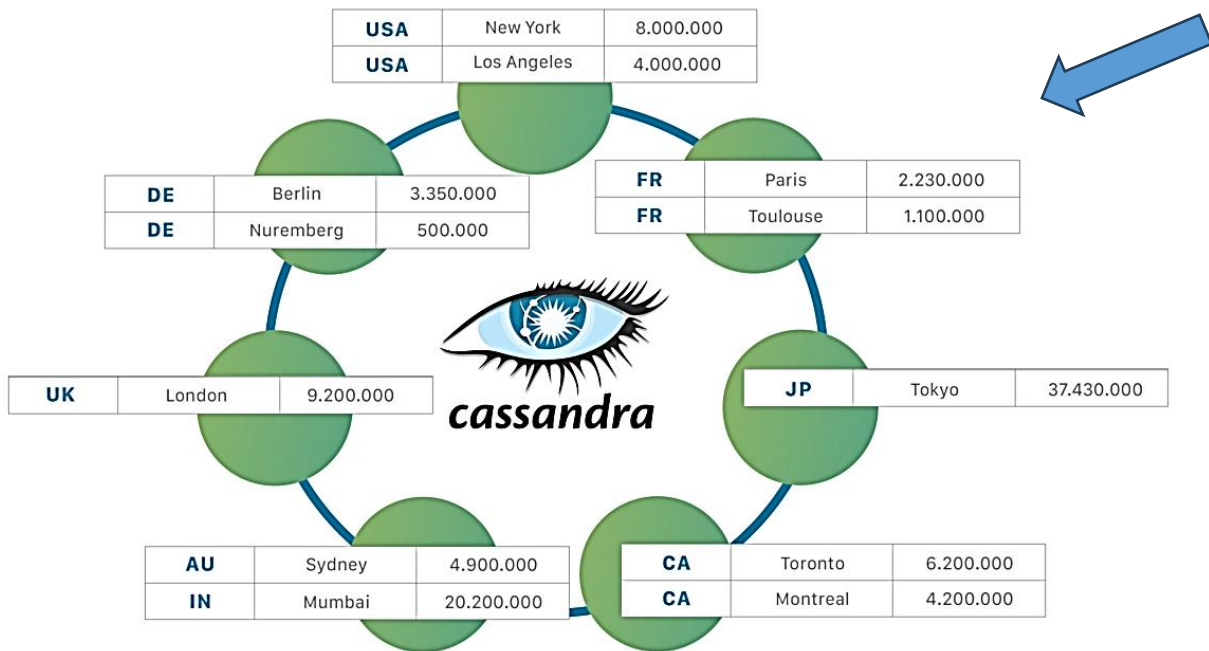
Es un contenedor de Keyspaces
que abarca uno o mas nodos

```
CREATE KEYSPACE mi_keyspace WITH  
replication = {  
  'class': 'SimpleStrategy',  
  'replication_factor': 3  
};
```



BD de Columna Ancha: Cassandra

Fragmentación Horizontal



COUNTRY	CITY	POPULATION
USA	New York	8.000.000
USA	Los Angeles	4.000.000
FR	Paris	2.230.000
DE	Berlin	3.350.000
UK	London	9.200.000
AU	Sydney	4.900.000
DE	Nuremberg	500.000
CA	Toronto	6.200.000
CA	Montreal	4.200.000
FR	Toulouse	1.100.000
JP	Tokyo	37.430.000
IN	Mumbai	20.200.000

Partition Key

BD de Columna Ancha: Cassandra

Fragmentación Horizontal

```
CREATE TABLE sensor_data
(
  Sensor int,
  Date date,
  Timestamp timestamp,
  Speed float,
  Torque float,
  Power float,
  PRIMARY KEY ((Sensor, Date), Timestamp)
) WITH CLUSTERING ORDER BY (Timestamp ASC);
```

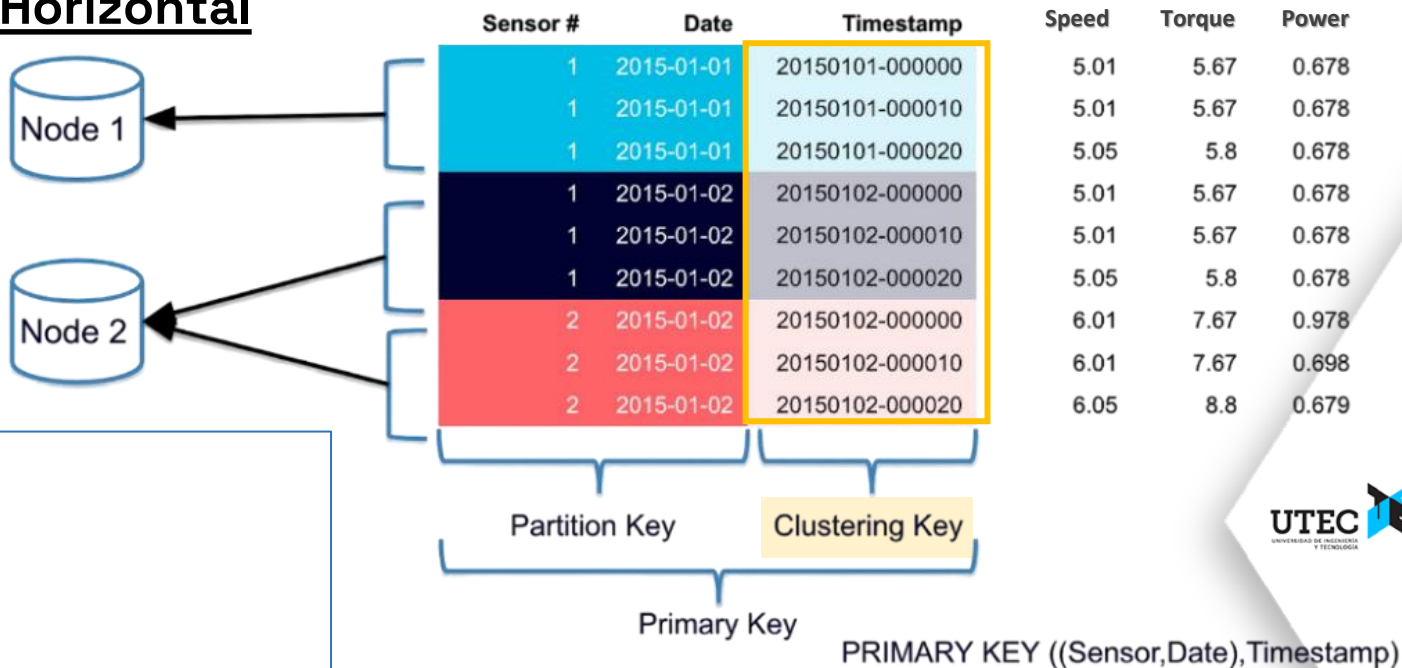
Primary Key = Partition Key + Clustering Key

Como los
datos están
distribuidos a
través de los
nodos

Como los
datos están
almacenados
en un nodo

BD de Columna Ancha: Cassandra

Fragmentación Horizontal



```
CREATE TABLE sensor_data
(
  Sensor int,
  Date date,
  Timestamp timestamp,
  Speed float,
  Torque float,
  Power float,
  PRIMARY KEY ((Sensor, Date), Timestamp)
) WITH CLUSTERING ORDER BY (Timestamp ASC);
```

BD de Columna Ancha: Ejemplo

Detección de fraude en pagos

Un sistema de pagos en línea quiere detectar transacciones sospechosas en tiempo real.

- Clave de **partición**: ID del usuario
- Clave de **clustering**: Fecha de la transacción.
- **Beneficios**: Se pueden construir sistemas de alerta en tiempo real altamente escalables para **consultas por rango**.

```
CREATE TABLE transacciones (  
    id_usuario UUID,  
    timestamp timestamp,  
    monto decimal,  
    ubicacion text,  
    metodo_pago text,  
    PRIMARY KEY (id_usuario, timestamp)  
) WITH CLUSTERING ORDER BY (timestamp DESC);  
  
-- Insertar transacción sospechosa (monto elevado):  
INSERT INTO transacciones (id_usuario, timestamp, monto,  
    ubicacion, metodo_pago)  
VALUES (uuid(), toTimestamp(now()), 5000.00, 'New York', 'Tarjeta  
de Crédito');  
  
-- Consultar transacciones en las últimas 24 horas  
SELECT * FROM transacciones WHERE id_usuario = <UUID>  
AND timestamp >= <TIMESTAMP_HACE_24_HORAS>;
```

- Las transacciones de **cada usuario se almacenan en la misma partición**.
- Dentro de esa partición, están **ordenadas por timestamp descendente** (útil para ver primero las más recientes).

5.



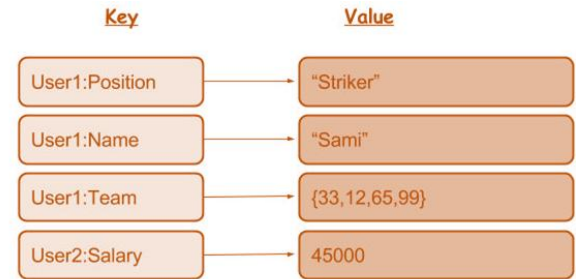
Base de Datos NoSQL

BD Clave - Valor

BD Clave-Valor

Características:

- Ideal para operaciones CRUD básicas
- No está diseñado para consultas complejas
- Los datos se organizan en una colección de pares clave-valor.
 - La clave debe ser única.
- Los valores pueden ser números enteros, cadenas o tipos de datos más complejos, como un documento JSON.
- Los datos se obtienen mediante una coincidencia exacta con la clave.



BD Clave-Valor

Casos de Uso:

- Operaciones CRUD básicas rápidas en datos no interconectados
- Almacenamiento de perfiles y preferencias de usuario en la aplicación
- Almacenamiento de sesiones de usuario
- Almacenamiento de datos del carrito de compras para tiendas en línea.
- Aplicaciones basadas en la ubicación (datos geoespaciales)



BD Clave-Valor: Productos

Base de Datos	Año de Creación	Empresa	Almacenamiento	Persistencia	Escalabilidad	Casos de Uso
Redis	2009	Redis Inc.	En memoria	Opcional (snapshot y AOF)	Alta (Cluster y Sentinel)	Caché, sesiones, colas de mensajes
DynamoDB	2012	Amazon Web Services (AWS)	SSD (gestionado)	Sí (gestionado por AWS)	Muy alta (serverless)	Aplicaciones web y móviles, IoT, gaming
Memcached	2003	Open Source	En memoria	No	Alta (sharding)	Caché distribuido, aceleración de bases de datos
Riak KV	2009	Open Source	Disco y memoria	Sí	Alta (replicación automática)	Aplicaciones distribuidas, big data
Berkeley DB	1991/2006	Oracle Corporation	Disco y memoria	Sí	Baja (embebida)	Aplicaciones móviles, dispositivos embebidos

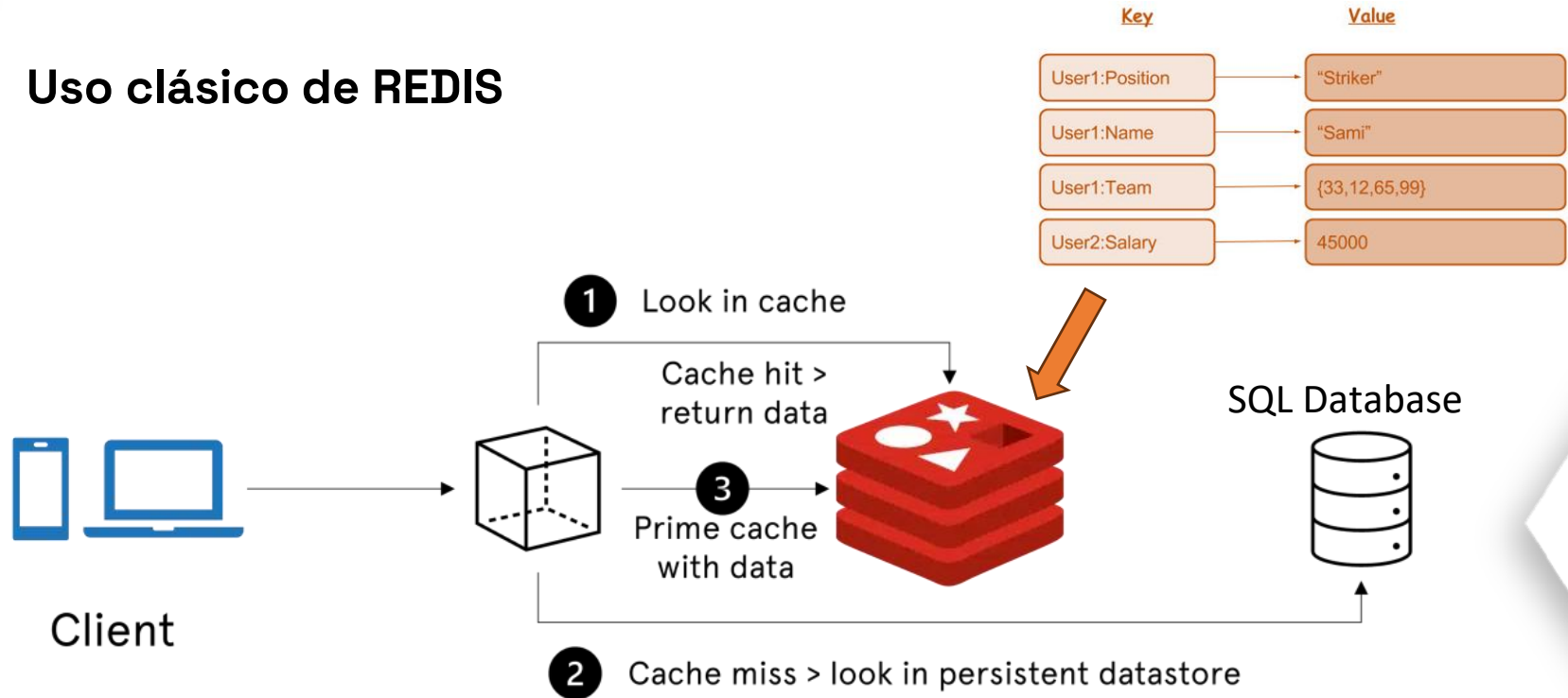
BD Clave-Valor: REDIS



- Redis sigue un modelo **clave-valor en memoria**, lo que le permite ofrecer una latencia extremadamente baja.
- Puede configurarse en **modo standalone**, **modo sentinel (replication)** o en **modo cluster (sharding)**.
- **Características clave:**
 - ✓ **Master-Slave Replication** para alta disponibilidad
 - ✓ **Cluster Mode** para escalabilidad horizontal
 - ✓ **Persistence (AOF y RDB)** para almacenamiento en disco

BD Clave-Valor: REDIS

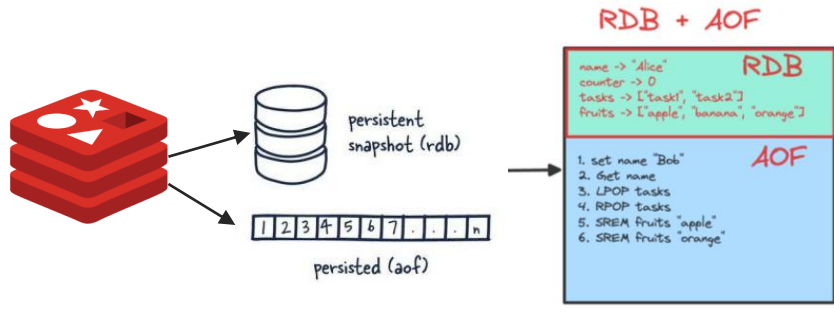
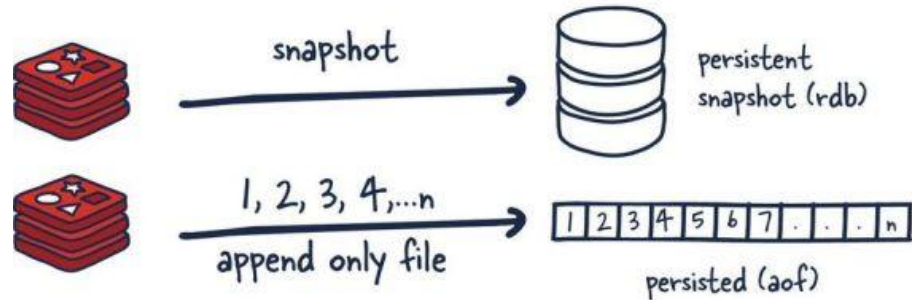
Uso clásico de REDIS



BD Clave-Valor: Persistencia en REDIS

Snapshot (RDB): copias puntuales del dataset en **intervalos configurables**.

Append Only File (AOF): registra cada operación de escritura en un archivo de log. **El archivo AOF puede crecer mucho.**



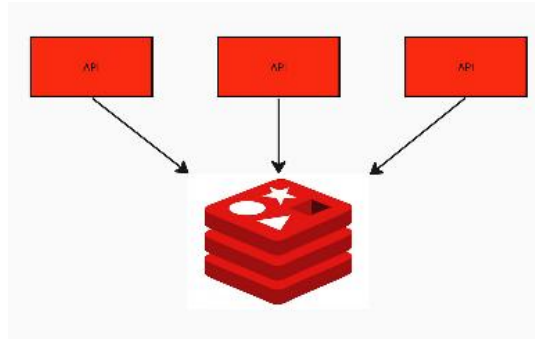
Modo híbrido: RDB + AOF

Combina rendimiento (RDB) y durabilidad (AOF).

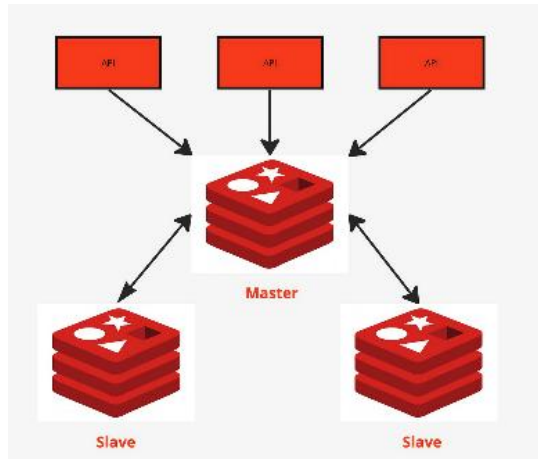
- Al iniciar Redis, primero se carga el ultimo RDB para restaurar el estado completo más reciente.
- Luego, se reproduce el archivo AOF para aplicar los cambios más recientes y llegar al estado actual.

BD Clave-Valor: Escalabilidad en REDIS

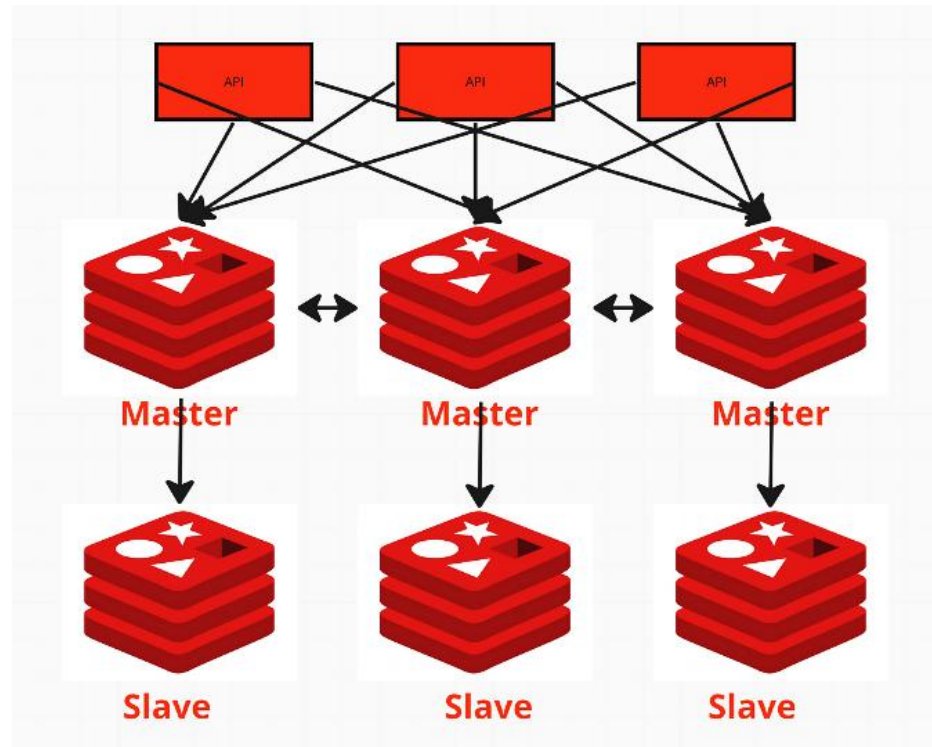
Redis Standalone



Redis Sentinel



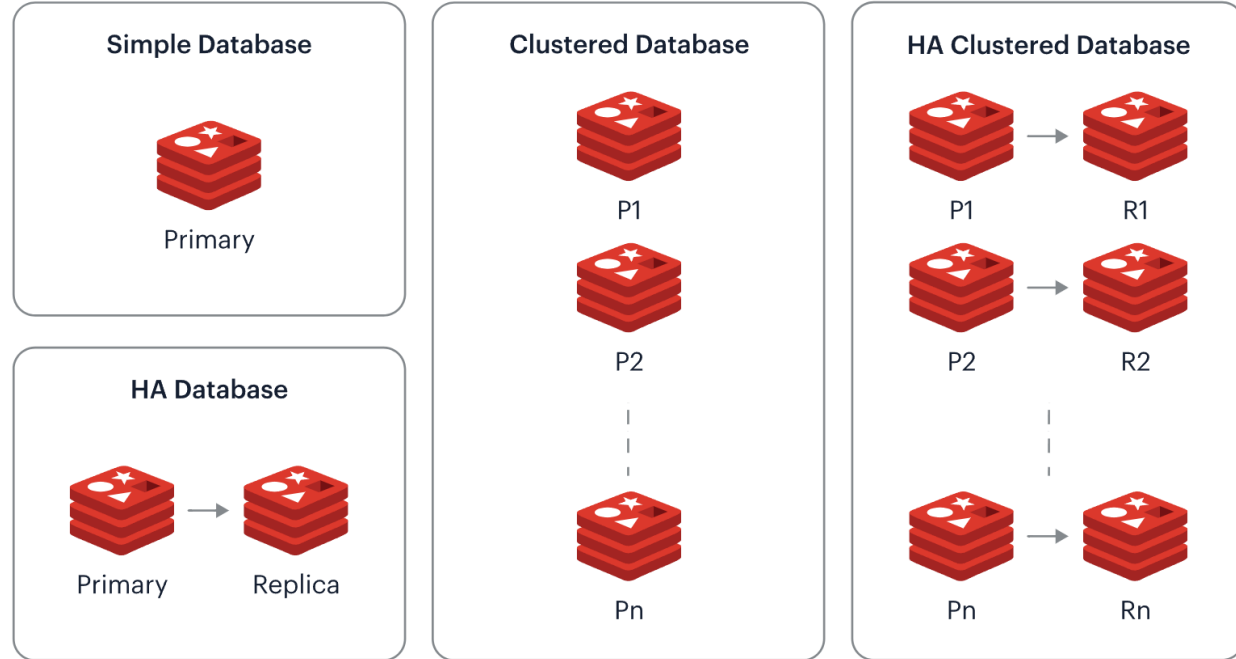
Redis Cluster



BD Clave-Valor: Escalabilidad en REDIS

- Shard automático.
- Rebalanceo sin interrupciones.
- Rendimiento extremo.

Redis Enterprise



BD Clave-Valor: Configuración de REDIS

- Instalación de REDIS con Docker

```
docker pull redis/redis-stack-server:latest
```

- Levantar el contenedor

```
docker run -d --name redis -p 6379:6379 redis/redis-stack-server:latest
```

The screenshot shows the Redis configuration interface with the 'Main' tab selected. The 'Server' section has the following settings: Type is set to 'Manual' (selected with a red arrow), Host is 'localhost' (indicated by a red arrow), Port is '6379' (indicated by a red arrow), Database is '0' (indicated by a red arrow), and Deployment is 'Standalone' (indicated by a red arrow). The 'Authentication' section shows 'Authentication' set to 'Database Native', 'Username' and 'Password' fields are empty, and the 'Save password' checkbox is checked. The 'Connection settings' section is partially visible at the bottom.

Opciones Online

<https://aws.amazon.com/es/elasticache/>

<https://cloud.redis.io/#/databases>

BD Clave-Valor: Operaciones en REDIS

Comandos CRUD:

1. **Crear (Create):** Utiliza el comando SET para almacenar un par clave-valor.

```
SET clave1 valor1
```

2. **Leer (Read):** Usa el comando GET para obtener el valor asociado a una clave.

```
GET clave1
```

3. **Actualizar (Update):** Redis no tiene un comando específico para actualizar. Puedes usar nuevamente SET para sobrescribir el valor de una clave existente.

```
SET clave1 nuevo_valor1
```

4. **Eliminar (Delete):** Usa el comando DEL para eliminar una clave.

```
DEL clave1
```

BD Clave-Valor: Operaciones en REDIS

Comandos CRUD en Python:

```
import redis

# Conectar a Redis
rc = redis.StrictRedis(host='localhost', port=6379, db=0)

# Crear (Insertar un par clave-valor)
rc.set('username:1', 'heider_cs')
rc.set('nombre:1', 'Heider Sanchez')
rc.set('pais:1', 'Perú')
rc.set('rol:1', 'Administrador')

# Leer (Obtener el valor de una clave)
nombre = rc.get('nombre')
print(f"Nombre: {nombre.decode('utf-8')}")

# Actualizar (Modificar el valor de una clave)
rc.set('rol', 'Operador')

# Eliminar (Eliminar un par clave-valor)
rc.delete('pais:1')
```

BD Clave-Valor: Operaciones en REDIS

TTL en REDIS

- TTL (Time To Live) es el tiempo de vida de una key en Redis antes de que sea eliminada automáticamente.
- Como se utiliza:
 - EXPIRE clave tiempo (en segundos)
 - PEXPIRE clave tiempo (en milisegundos)
 - TTL clave (para conocer el tiempo restante)
 - PERSIST clave (para que la clave no expire)

```
# Crear una clave con un TTL de 10 sec.
```

```
rc.setex("mi_clave", 10, "mi_valor")
```

```
# Comprobar el TTL de la clave
```

```
ttl = rc.ttl("mi_clave")
```

BD Clave-Valor: Ejemplo 1

Caché de respuestas de API

- Se sabe que el servicio de clima actualiza los datos cada 10 minutos.
- Con REDIS almacenamos en caché por 10 minutos para reducir llamadas innecesarias.
- **Beneficios:** Menos peticiones a la API externa, reducción de costos y menor tiempo de respuesta para el usuario.

```
def obtener_clima(ciudad):  
    cache_key = f"clima:{ciudad}"  
  
    # Intentar obtener desde Redis  
    clima = rc.get(cache_key)  
    if clima:  
        return json.loads(clima)  
  
    # Si no está en caché, llamar a la API  
    url = f"{URL_WEATHERAPI}?key={API_KEY}&q={ciudad}"  
    response = requests.get(url)  
    data = response.json()  
  
    # Guardar en Redis con una expiración de 10 minutos  
    rc.setex(cache_key, 600, json.dumps(data))  
  
    return data
```

BD Clave-Valor: Ejemplo 2

Caché de autenticación de usuarios

Un sistema web donde se evita la verificación repetitiva de credenciales en la base de datos.

- Con REDIS el token de sesión en caché y se renueva automáticamente.
- **Beneficios:** Evita múltiples validaciones de token en la base de datos, reduciendo la carga del sistema.

```
def guardar_sesion(usuario_id, token):  
    # Expira en 1 hora  
    rc.setex(f"sesion:{usuario_id}", 3600, token)  
  
def verificar_sesion(usuario_id, token):  
    token_guardado = rc.get(f"sesion:{usuario_id}")  
    if token_guardado and token_guardado.decode() ==  
token:  
        return True  
  
# Si no hay token en Redis, renovar con un Refresh Token.  
refresh_token = get_refresh_token_de_bd(usuario_id)  
if refresh_token:  
    nuevo_token = generar_nuevo_token(usuario_id)  
    guardar_sesion(usuario_id, nuevo_token)  
    return nuevo_token  
  
return False
```

BD Clave-Valor: Comandos REDIS

Comandos Básicos

Comando Redis	Equivalente en Python	Ejemplo en Python
SET clave valor	<code>r.set(clave, valor)</code>	<code>r.set("nombre", "Carlos")</code>
GET clave	<code>r.get(clave)</code>	<code>r.get("nombre") → "Carlos"</code>
DEL clave	<code>r.delete(clave)</code>	<code>r.delete("nombre")</code>
EXISTS clave	<code>r.exists(clave)</code>	<code>r.exists("nombre") → 1 (True)</code>
EXPIRE clave segundos	<code>r.expire(clave, segundos)</code>	<code>r.expire("nombre", 10)</code>
TTL clave	<code>r.ttl(clave)</code>	<code>r.ttl("nombre")</code>

BD Clave-Valor: Comandos REDIS

Trabajando con Listas

Comando Redis	Python	Ejemplo
LPUSH lista valor	<code>r.lpush(lista, valor)</code>	<code>r.lpush("tareas", "Estudiar")</code>
RPUSH lista valor	<code>r.rpush(lista, valor)</code>	<code>r.rpush("tareas", "Dormir")</code>
LPOP lista	<code>r.lpop(lista)</code>	<code>r.lpop("tareas")</code>
RPOP lista	<code>r.rpop(lista)</code>	<code>r.rpop("tareas")</code>
LRANGE lista inicio fin	<code>r.lrange(lista, inicio, fin)</code>	<code>r.lrange("tareas", 0, -1)</code>

BD Clave-Valor: Comandos REDIS

Trabajando con Hashes (Diccionarios)

Comando Redis	Python	Ejemplo
HSET hash campo valor	<code>r.hset(hash, campo, valor)</code>	<code>r.hset("usuario:1", "nombre", "Juan")</code>
HGET hash campo	<code>r.hget(hash, campo)</code>	<code>r.hget("usuario:1", "nombre")</code>
HGETALL hash	<code>r.hgetall(hash)</code>	<code>r.hgetall("usuario:1")</code>
HDEL hash campo	<code>r.hdel(hash, campo)</code>	<code>r.hdel("usuario:1", "nombre")</code>

BD Clave-Valor: Comandos REDIS

Trabajando con Conjuntos (Sets)

Comando Redis	Python	Ejemplo
SADD set valor	<code>r.sadd(set, valor)</code>	<code>r.sadd("colores", "rojo")</code>
SMEMBERS set	<code>r.smembers(set)</code>	<code>r.smembers("colores")</code>
SREM set valor	<code>r.srem(set, valor)</code>	<code>r.srem("colores", "rojo")</code>

BD Clave-Valor: Comandos REDIS

Trabajando con Sorted Sets (ZSET)

Comando Redis	Python	Ejemplo
ZADD ranking score valor	<code>r.zadd(ranking, {valor: score})</code>	<code>r.zadd("ranking", {"Carlos": 100})</code>
ZRANGE ranking 0 -1 WITHSCORES	<code>r.zrange(ranking, 0, -1, withscores=True)</code>	<code>r.zrange("ranking", 0, -1, withscores=True)</code>

Laboratorio 15

